

Typed Decisions in Agent Memory: Where They Help, Where They Don't, and What It Costs

Rishabh Sharma

ORCID: [0000-0000-0000-0000]

Independent researcher

[email@domain]

Abstract

Typed decisions cut an agent memory system's decision cost and latency and improve retrieval under a small budget; they do not change answers after facts change, and a small model used zero-shot cannot make them. Memory systems make an LLM call for every write decision, which makes revisiting the store unaffordable, yet most of that work is choice among fixed options. In engram, an LLM extracts facts and every decision is a typed question answered by a hosted decision model (Jev). With extraction held identical to mem0 2.1.0's, typed decisions cut decision-layer cost by $70.0\times$ and median decision latency by $27.6\times$, with equal dev accuracy (31/35 against 31/35). On 610 held-out LoCoMo questions under a three-memory retrieval budget, engram answers +8.7 points more than mem0 at matched context tokens (95% CI +5.2 to +12.1); at $k=20$ the systems tie (+0.8, CI -2.3 to +3.9). A belief-state store policy over-closed no labeled keep item. Two results are negative: closing stale facts does not change answers on current benchmarks, and the 421M base checkpoint of the open-weights decision model Laya, used zero-shot as its documentation advises against, does not make the relational decisions.

1 Introduction

The cost problem. A memory system for an LLM agent turns conversation into facts it can retrieve later. Each write has two parts. Extraction asks what facts a message states. Decisions ask, for each fact, whether it is new, a duplicate, a refinement or a change to something already stored, whether it is worth keeping, and whether it is sensitive. In current open-source systems both parts are LLM calls. mem0 2.1.0's default `add()` makes a single LLM call per message with its additive extraction prompt, and has no separate update or delete step (mem0/memory/main.py, `Memory._add_to_vector_store`). An LLM call per decision is ex-

pensive enough that nothing re-examines the store afterwards. Facts that stopped being true stay in it.

The observation. The decisions are choices among options fixed in advance. That is the setting typed decision models are built for. They return a probability over a fixed option set in one short request, and they cost far less than generating text. engram is built on that observation: an LLM extracts facts, and every decision after extraction is a typed question (Figure 1).

Concurrent work. The architectural idea of routing memory decisions to a typed decision model was reached independently by Jev-Mem (Jiang et al., 2026a), which uses Jev for typing, relation construction, query routing, budget allocation, traversal, candidate scoring and stopping. It reports a LoCoMo judge score of 0.777 with a 158 s build and 0.93 s query against A-MEM (Xu et al., 2025), MAGMA (Jiang et al., 2026b) and two further systems reported in that paper. It does not isolate the decision layer, evaluate updates or closes, measure calibration, or use a held-out split or confidence intervals. Separately, a community article reranked AtMem's top-10 with Jev on 1,986 LoCoMo questions and measured the effect at the ranking level: `MRR@5` rose from 0.4259 to 0.5868 and `Recall@1` from 0.3399 to 0.5423 (Taghia, 2026). This paper's contribution is the controlled measurement: an identical-extraction ablation of the decision layer, its calibration, the safety of the store it drives, the rerank effect on answers with a matched-context control, and the negatives.

Contributions. The paper makes four. First, with extraction held identical, typed decisions replace an LLM decision layer at $70.0\times$ lower cost and $27.6\times$ lower median latency with no measured accuracy loss (§5.1). Second, a belief-state store policy with reversible, gated closes over-closes no labeled keep item, and its v3 rule removes a weak-evidence failure that closes true facts (§3.3, §5.3).

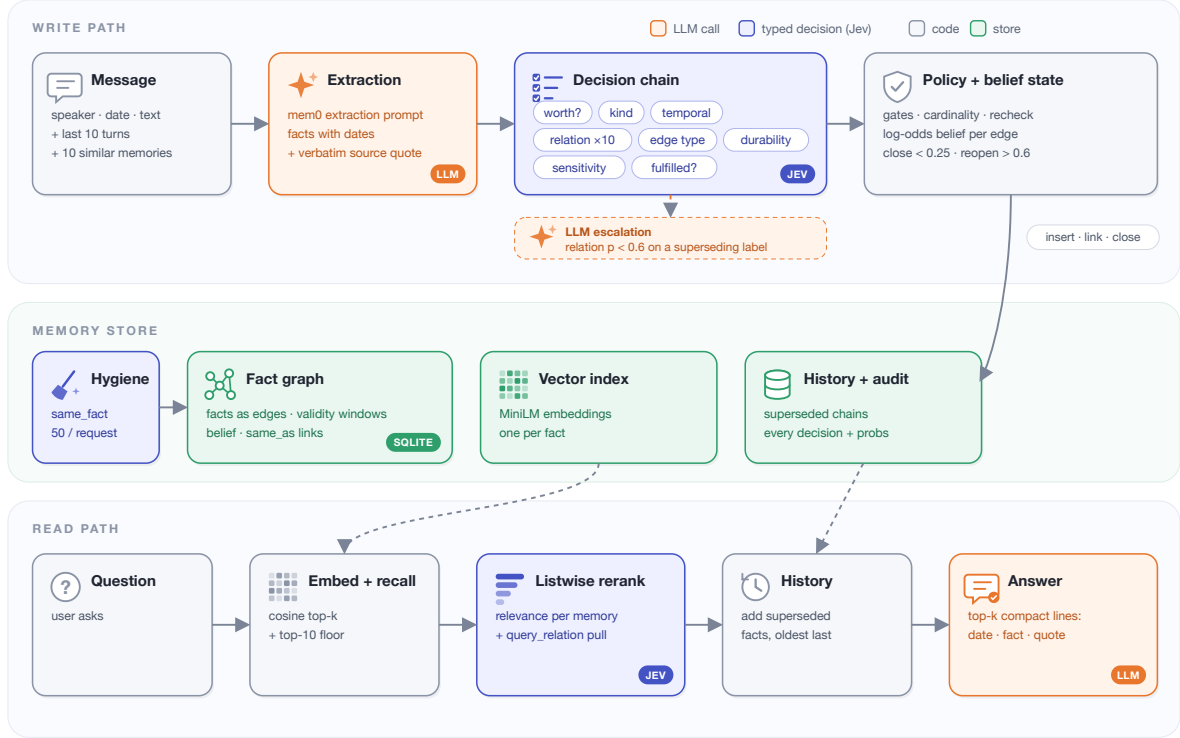


Figure 1: engram’s write and read paths. Orange boxes are LLM calls, blue boxes are typed Jev decisions.

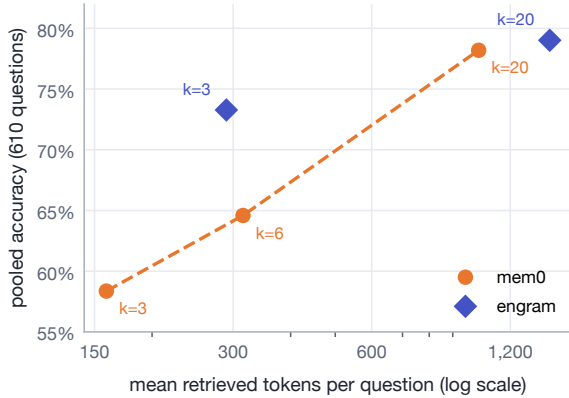


Figure 2: Pooled held-out accuracy (610 questions) against mean retrieved tokens per question. mem0 accuracy was measured at three settings, $k=3, 6$ and 20 ; the token-matching sweep counted tokens at the other k without answering. Models as in Table 2.

Third, under a three-memory budget a listwise Jev rerank puts engram +8.7 points ahead of mem0 at matched context on 610 held-out questions, while at $k=20$ the systems tie (§5.2). Fourth, two results are negative: closing stale facts does not change answers on current benchmarks, and a zero-shot base checkpoint of an open-weights decision model does not make the relational decisions (§5.4). Figure 2 plots the third against the context each system shows the answer model.

Scope. We compare against one baseline (mem0 OSS 2.1.0) on one benchmark (LoCoMo: one development conversation and four held-out conversations, scoring four of its five question categories; adversarial is excluded, following mem0’s evaluation protocol). We use update sets written by the system’s author, and one hosted decision model (Jev, jev-1.13.0). We did not test Zep/Graphiti or Letta, LongMemEval, other extraction, answer or judge models, multi-user stores, or non-English text.

2 Background and Related Work

2.1 Write decisions in existing systems

mem0 (Chhikara et al., 2025) extracts and consolidates facts with LLM calls. Version 2.1.0 extracts with one LLM call per message; the prompt receives the new message, the last 10 messages and the 10 most similar existing memories, and emits only additions. Its older update step, which asked an LLM to label each new fact as ADD, UPDATE, DELETE or NONE against existing memories, is still shipped as DEFAULT_UPDATE_MEMORY_PROMPT. We use it as the LLM decision layer in §5.1.

Other systems also decide with an LLM. Zep’s Graphiti builds a temporal knowledge graph and resolves entities and invalidates edges with LLM

calls (Rasmussen et al., 2025). MemGPT, now Letta, manages memory tiers through LLM function calls (Packer et al., 2023). A-MEM links notes with LLM-written attributes (Xu et al., 2025), and MAGMA organizes memory as multiple graphs (Jiang et al., 2026b). ByteRover curates a hierarchical context with an LLM; its retrieval is a five-tier progressive strategy that answers most queries in under 100 ms without LLM calls and escalates to agentic reasoning only for novel questions (Nguyen et al., 2026).

2.2 Typed decision models

A typed decision model takes a shared *state* (JSON) and a set of questions, and returns a probability distribution for each question. A question is a choice among named options, each with a rubric; a yes/no (“noul”) question; or a score.

Jev. We use Jev through TypeSafe’s API: model `jev-1.13.0`, priced at 0.042 USD per million input tokens as billed to our account (`src/engram/config.py`); the public documentation does not list a price. Its documentation gives a limit of 255 options per choice question (TypeSafe AI, 2026). Measured latency is flat in request size (§5.5).

Laya. Laya is an open-weights model with the same request format. We ran checkpoint `convaiinnovations/laya` locally through the `laya-mlx` port on an Apple M2 Pro with 32 GB. It reads at most 512 tokens per question, of which at most 192 go to the instructions and options. This is the 421M base checkpoint. Its model card reports zero-shot typed-decision accuracy of 0.362 against a 0.318 random baseline, calls it “a fast base to specialise, not a zero-shot decision engine,” and offers a separate checkpoint fine-tuned for typed decisions (Convai Innovations, 2026). We used the base checkpoint zero-shot.

2.3 LoCoMo and its limits

LoCoMo (Maharana et al., 2024) contains long multi-session conversations with questions in five categories, of which we score four (adversarial is excluded, following `mem0`’s evaluation protocol; §4.1). LongMemEval (Wu et al., 2025) also evaluates long-term conversational memory; we used only LoCoMo.

It barely tests updates. In the first 215 messages of `conv-26`, an LLM labeler (`claude-sonnet-4-6`, prompt in `bench/stale.py`) found 2 claims that a later message makes no longer true (`bench/slices/conv26_superseded.json`).

Public scores are not comparable. Published LoCoMo scores for the same systems differ between the systems’ own reports and third-party reports (Mem0, 2026; ByteRover, 2026). We do not quote those numbers. All comparisons here run both systems under one protocol.

2.4 Closest work and adjacent ideas

Jev-Mem and AtMem. Jev-Mem (Jiang et al., 2026a) is the closest design: Jev controls typing, relation construction, query routing, budget allocation, traversal, candidate scoring and stopping. The two designs differ in the store and in retrieval. Jev-Mem runs with admission filtering off and preserves every observation, so its store is never updated or closed; engram’s store is governed by a close/belief policy with reversible closes and merges (§§3.2–3.3). Jev-Mem routes queries across multiple views; engram uses a single listwise rerank over cosine candidates plus a query-relation pull (§3.1). The AtMem–Jev article (Taghia, 2026) measured the rerank at the ranking level (Recall@10 unchanged; median batch latency 3.32 s) and reported no answer accuracy or intervals.

Reranking and context. Long contexts are used poorly by language models (Liu et al., 2024), and LLMs rerank candidates well (Sun et al., 2023); spending compute on selecting context rather than adding more of it follows from both, and our rerank result is an instance.

Routers. Routers send queries to cheaper models (Ong et al., 2025; Chen et al., 2024), and small classifiers act as guardrails (Inan et al., 2023). The escalation rule of §3.4 is a router of that kind.

Calibration. Temperature scaling (Guo et al., 2017) and conformal prediction (Angelopoulos and Bates, 2021) give principled thresholds for acting on a model’s probability. The belief update of §3.3 depends on the former.

3 engram

3.1 Pipeline

Figure 1 shows both paths.

Write path. An LLM extracts facts from a message; in the experiments it uses `mem0`’s prompt and inputs (§4.2). Each fact then goes to Jev as a single request that carries the fact questions and one `relation_to_candidate` question for each of up to 10 candidate facts. A policy layer and a belief state (§§3.2–3.3) turn the answers into store opera-

tions. The store is SQLite plus NetworkX: facts are edges with validity windows, and every decision is logged with its probabilities.

Read path. The question is embedded and the cosine top-k taken. Jev reranks these candidates with one yes/no relevance question per candidate, plus a query_relation question that can pull in facts by relation type. History is added next, meaning the facts that each retrieved fact superseded. The answer model sees compact lines, each with the date the fact was said, the fact, and the verbatim source quote.

3.2 Decision chain and policy layer

The current chain has 12 questions, listed in [Appendix A](#). Choice questions carry a rubric per option. When a question’s wording changes, its version number is bumped and old arms keep the old version.

Policy. Decisions act only through explicit rules (`src/engram/pipeline/write.py`). A decision acts at $p \geq 0.85$. A superseding relation (update, contradiction, negates) below 0.60 is escalated to an LLM with `mem0`’s update prompt, and anything in between is stored as tentative.

Closing an edge is gated three ways. The new fact must be current or past, with $P(\text{current}) + P(\text{past}) \geq 0.85$, so planned and hypothetical statements never close anything. The relation type’s cardinality must allow the close: contradiction closes only a sibling (same subject and relation) on a single-valued relation, update may also close a multi-valued sibling, and negates (the new fact says the old one stopped) may close any edge. And the first piece of evidence against an edge counts only if a second phrasing of the relation question (`relation_to_candidate_recheck`) confirms it.

Three further rules cover plans, duplicates and overrides. A dedicated yes/no question, `plan_fulfilled`, asks whether a new fact reports that a stored plan has happened; at $p \geq 0.85$ the plan closes with reason *fulfilled*. A duplicate becomes a `same_as` link that is collapsed at retrieval and can be removed, so text is never merged. An explicit request to remember overrides `worth_remembering`, and a credential is redacted before storage whatever the other answers say.

Hygiene. After ingestion, one pass re-checks every group of facts that share a subject and relation. It asks the yes/no question `same_fact` for each pair, 50 pairs per request. At $p \geq 0.85$ the pair is linked, and at $p \leq 0.15$ an existing link is

dropped. On the held-out conversations one pass made between 1,865 and 5,446 decisions, for between \$0.0112 and \$0.0332 ([§5.3](#)). With LLM decisions, re-examining the store at that scale is what becomes unaffordable.

3.3 Belief state

Each edge i carries a belief $b_i \in [0.02, 0.98]$ that it is currently true. A typed answer z about the edge, with probability q for its chosen label, updates it in log-odds:

$$\text{logit } b_i \leftarrow \text{logit } b_i + w(z) \text{ logit } q, \quad (1)$$

$$\text{logit } q \leftarrow \text{logit } q / T. \quad (2)$$

With a per-question temperature T ([§5.3](#)), $\text{logit } q$ is first rescaled as in [Eq. \(2\)](#). We use $|w| = 1$, and the sign follows the policy rules of [§3.2](#): $w = +1$ for the supporting answers duplicate and refinement, $w = -1$ for update, contradiction and negates where a close is permitted, and $w = 0$ for new and for any answer a gate blocks.

An open edge closes when $b_i < 0.25$ and reopens when $b_i > 0.60$. The gap between the two gives hysteresis.

Belief v3. The v2 rule used in the held-out runs lets any answer count. A duplicate at $q < 0.5$ therefore has $\text{logit } q < 0$ and *lowers* belief, and a weak against-answer raises it. v3 counts an answer only when its label is the argmax and $q > 0.5$ ([§5.3](#)). [Figure 3](#) traces one fact from the dev run under both rules.

What this assumes. The log-odds rule [Eq. \(1\)](#) treats q as a calibrated likelihood. Where the model is miscalibrated, belief moves by the wrong amount. This is why [§5.3](#) measures calibration and why a per-question temperature is part of the rule.

3.4 Cost model

With a per-decision Jev cost c_J , an LLM escalation cost c_L and an escalation threshold θ on the top probability q :

$$C(\theta) = c_J + P(q < \theta) c_L, \quad (3)$$

$$E(\theta) = P(q \geq \theta) \varepsilon_J(\theta) + P(q < \theta) \varepsilon_L, \quad (4)$$

where $\varepsilon_J(\theta)$ is Jev’s error rate on the decisions it keeps and ε_L is the LLM’s error rate on the escalated ones. [§5.3](#) draws the empirical curves of [Eqs. \(3\) and \(4\)](#).



Figure 3: Belief in the fact “Melanie carves out daily me-time through running, reading, or playing violin” (message D2:5), dev + update set 1, under v2 and v3. Under v2 a weak refinement answer at D2:7 ($p = 0.45$) lowers belief slightly, so the update at U:E11 takes it below the close line; v3 ignores that answer (×) and the fact closes one message later, at U:S01. Source: the belief trace in `bench/results/e4_belief_v2__dev_updates__k3__noanswer.json` and its v3 counterpart.

4 Experimental Setup

4.1 Models, data and protocol

Models. Every arm uses the same models. `claude-haiku-4-5` extracts facts, for `engram` and for `mem0 2.1.0` (which runs with telemetry off). `claude-sonnet-4-6` answers and judges, with `mem0`’s LoCoMo evaluation prompts (§4.2), and is also the LLM decision layer and the escalation model, with `mem0`’s update prompt.

Data. The *dev* slice is conv-26 sessions 1–4 (76 messages, 35 questions), and the *stress* slice is conv-26 sessions 1–10 (215 messages, 80 questions). The *held-out* slice is conv-30, conv-41, conv-42 and conv-43 whole (610 questions); none of it was used during development, and the system configuration was frozen (git tag `e4-frozen`) before any held-out run. LoCoMo’s questions fall in five categories, of which we score four (adversarial is excluded, following `mem0`’s evaluation protocol). Each held-out conversation is ingested once per system, and $k=3$ and $k=20$ are answered from the same store.

Caching and budget. Every LLM and Jev call is cached by its full request, and budget guards stop any run past a spending limit. Phase 2 (all experiments reported here) spent \$93.26 over 80 ledgered runs. Jev accounts for \$1.47 of it (`bench/results/phase2_spend.jsonl`; per-arm totals in Appendix D). The build phase was not ledgered;

roughly \$10 by the author’s estimate.

4.2 Identical extraction

For each message, `mem0 2.1.0`’s `add()` builds its extraction prompt from the new message (as `[date] speaker: text`), the last 10 messages, the 10 existing memories most similar to it, and a current date. It passes no observation date, so relative dates in historical conversations resolve against the current date. We report this as-is, and a patched variant in §6.

`engram`’s E-arms build the same prompt with the same function (`generate_additive_extraction_prompt`), inputs and model (`bench/run.py`, `src/engram/flags.py`). The “existing memories” differ only because each system’s store differs. The E2 arms differ only in what decides after extraction.

Prompts. `mem0`’s extraction and update prompts, and the functions that build their requests, are copied verbatim from the installed `mem0 2.1.0` package (Apache License 2.0) into `src/engram/llm/prompts_mem0.py`. `tests/test_prompts_mem0.py` checks that the copies are byte-identical to the installed package, so a changed prompt fails the test suite. The answer and judge prompts are `mem0`’s LoCoMo evaluation prompts, adapted from two per-speaker memory lists to a single list (`bench/locomo_subset.py`).

4.3 Update sets

LoCoMo has almost no updates (§2.3). Two update sets, written in the speakers’ voices, extend it. Both are appended to the dev slice and were frozen by SHA-256 before any run.

Set 1 (`bench/updates_conv26.json`, SHA-256 prefix `2330876388f1`) has 30 items: 15 easy closes, 10 subtle items labeled *no_close* (past-tense mentions and unrealised plans that must not close anything), and 5 fulfilled plans. Set 2 (`bench/updates2_conv26.json`, prefix `d4f3c8d2de61`) adds 28 messages and 20 questions. Of these, 5 ask about a past moment in set 1’s items, 5 ask for the current value after a chain of 3–5 changes, 5 ask for a chain’s value at a past moment, and 5 follow an update whose message has no temporal cue such as “now” or “anymore”. Appendix B shows one item of each kind; the full sets are in the two files. The system’s author wrote and labeled them, and this is a source of bias.

4.4 Statistics

Both systems answer the same questions, so comparisons are paired: $d_i = \text{engram}_i - \text{mem0}_i$. We report an exact McNemar test on the discordant questions and a 95% interval from the per-question normal approximation. We also report a cluster bootstrap that resamples the four held-out conversations; with four clusters it is a robustness check, not a primary interval. The token-matched comparison reports the per-question interval only.

5 Results

5.1 Replacing the decision layer

The three arms in Table 1 share extraction. The two engram arms differ only in what decides after it: Jev’s typed questions in E2 Jev, and claude-sonnet-4-6 with mem0’s update prompt, one call per extracted fact, in E2 LLM.

The two decision layers answer the same number of questions and mem0 one fewer, within noise. The Jev decision layer costs $70.0\times$ less and has $27.6\times$ lower median decision latency. The LLM decision layer stores fewer facts (18 against 47) because mem0’s UPDATE event rewrites an existing memory instead of adding one.

The two latency columns are medians over different messages. In the E2 LLM arm only 30 of 76 messages made an LLM decision call, so the write median falls on an extraction-only message (median extraction 869 ms); on the messages with decisions, the logged median of the slowest decision call is 7,892 ms (bench/e2_latency.py).

In Table 1 the decision layer is 1.3% of the Jev arm’s end-to-end write cost and 47.0% of the LLM arm’s; with typed decisions, extraction is almost the whole cost of a write.

These ratios compare Jev against claude-sonnet-4-6 as the decider. We have no measurement with a smaller LLM decider. On the stress slice the Jev arm scored 67/80 and mem0 64/80; the LLM arm was not run there.

Held-out parity at default k. The held-out runs compare the frozen full system (E4 belief v2, §3.3) with mem0, not the E2 arms. At $k=20$ engram answers 482/610 and mem0 477/610. The difference is +0.8 points (95% CI -2.3 to +3.9; conversation bootstrap -4.5 to +5.4; McNemar $p = 0.679$), within noise.

5.2 Retrieval under a small budget

Our answer-level result is consistent with the ranking-level improvement AtMem measured when reranking with Jev (Taghia, 2026).

At $k=3$ engram shows the answer model 290 tokens per question and mem0 159. Most of the difference is the source quote on each engram line. To separate context size from ranking, we answered the same 610 questions from mem0’s existing held-out stores at every k from 3 to 8. The token-matched setting is the k whose mean retrieved tokens came closest to engram’s 290: $k=6$, at 315 tokens. That gives mem0 slightly more context than engram. The run cost \$2.52.

At $k=3$ engram is ahead of mem0 by +14.9 points (95% CI +11.2 to +18.7). Against token-matched mem0 the difference is +8.7 points (95% CI +5.2 to +12.1; McNemar $p = 1.3e-06$; 86 questions only engram answered against 33 only mem0 answered).

Figure 2 places the three measured mem0 settings and the two engram settings on one axis of retrieved tokens. engram at $k=3$ sits above the line through mem0’s points, and the two systems meet at $k=20$.

engram’s point estimate is ahead in every category at matched context (Table 3) and in every conversation (Appendix C). Per conversation, the matched-context interval excludes zero in 2 of four conversations (conv-42 and conv-43); conv-30 and conv-41 are within noise on their own.

Attribution. Matching context removes 41.8% of the $k=3$ difference. The other 58.2% is what remains once context is matched.

We attribute that remainder to which memories reach the top three, which rests on a dev-slice ablation, not a held-out one. On dev + set 1 at $k=3$ the frozen arm answered 30/30 update questions. It answered 26/30 with Jev reranking off, 30/30 with history expansion off, and mem0 answered 25/30. We did not run a held-out no-rerank arm.

5.3 Store safety and calibration

Figure 4 shows the same outcomes per arm.

Over-closes. No labeled keep item was over-closed by any arm.

Closes outside the labels. These separate the policies. The first Jev arm (E2), which closed whenever a superseding label cleared 0.85, made 11 closes on dev + set 1, and 8 of them match no labeled pair. Belief v2 made 4, of which 1 match

Table 1: Dev slice, conv-26 sessions 1–4, 76 messages, 35 questions, all retrieved memories (default k). Extraction claude-haiku-4-5 for all arms; answers and judge claude-sonnet-4-6; E2 LLM decides with claude-sonnet-4-6. Costs are per 1,000 messages written. Decision p50: median time after extraction, per message, over messages that produced at least one fact. Write p50: median end-to-end write time per message over all messages, including those that produced no facts.

system	accuracy	decision \$/1k	decision p50	total \$/1k	write p50	facts
mem0 2.1.0 (one LLM call per write)	30/35	–	–	\$9.80	879 ms	46
engram E2, LLM decision layer (mem0 update prompt)	31/35	\$8.782	7,675 ms	\$18.70	918 ms	18
engram E2, Jev decision layer	31/35	\$0.125	278 ms	\$9.90	895 ms	47

Table 2: Held-out LoCoMo accuracy, pooled over conv-30, 41, 42 and 43 (610 questions, adversarial category excluded), with each mem0 row’s paired difference against engram at the same budget (engram $k=3$ for the $k=3$ and token-matched rows, engram $k=20$ for the $k=20$ row). Extraction claude-haiku-4-5, answers and judge claude-sonnet-4-6; one ingestion per system per conversation. Tokens: mean retrieved-context tokens per question. 95% CI: per-question normal approximation; bootstrap: resampling the four conversations. Discordant: questions only engram / only mem0 answered correctly.

system	k	tokens/q	accuracy	Δ (pts)	95% CI	95% CI, bootstrap	discordant	McNemar p
engram	3	290	73.3%	–	–	–	–	–
mem0	3	159	58.4%	+14.9	[+11.2, +18.7]	[+8.2, +19.8]	120 / 29	2.4e-14
mem0 (token-matched)	6	315	64.6%	+8.7	[+5.2, +12.1]	hot computed	86 / 33	1.3e-06
engram	20	1,461	79.0%	–	–	–	–	–
mem0	20	1,024	78.2%	+0.8	[-2.3, +3.9]	[-4.5, +5.4]	49 / 44	0.679

Table 3: Held-out accuracy by LoCoMo category, 610 questions, k as labeled. Models as in Table 2.

category	Q	engram $k=3$	mem0 $k=3$	mem0 $k=6$ (matched)	engram $k=20$	mem0 $k=20$
multi-hop	110	49.1%	31.8%	40.9%	67.3%	61.8%
temporal	119	84.0%	71.4%	73.9%	88.2%	84.0%
open-domain	33	51.5%	30.3%	33.3%	51.5%	57.6%
single-hop	348	79.3%	64.9%	71.8%	82.2%	83.3%

Table 4: Store safety and update-question accuracy on the dev slice with update sets 1 and 2 (extraction claude-haiku-4-5, decisions Jev jev-1.13.0, answers and judge claude-sonnet-4-6). Closes are on dev + set 1, split by whether (closed fact’s message, closing message) is a labeled update or superseded pair. Over-closed: set-1 no_close items whose fact was closed by their own update message, over those stored. Stale: close items whose old fact is still active, over close items stored. Every arm kept the one set-2 keep item. Accuracy is at default k (all retrieved memories) unless marked; “–”: run without answering or not run.

arm	closes	labeled	unlabeled	over-closed	set-1 stale	set-2 stale	set-1 acc.	set-1 acc. $k=3$	set-1 acc. no dates	set-2 acc.
mem0 (add-only)	0	0	0	0/7	18/18	16/16	29/30	25/30	29/30	20/20
E2: Jev, replace-on-update	11	3	8	0/8	10/18	15/16	30/30	30/30	30/30	19/20
E3: structural rules	3	2	1	0/8	17/18	not run	–	–	–	–
E4: belief v1	3	2	1	0/8	17/18	15/16	30/30	30/30	29/30	19/20
E4: belief v2 (frozen)	4	3	1	0/8	14/18	10/16	30/30	30/30	–	20/20
E4: belief v3	4	2	2	0/8	14/18	10/16	–	–	–	–

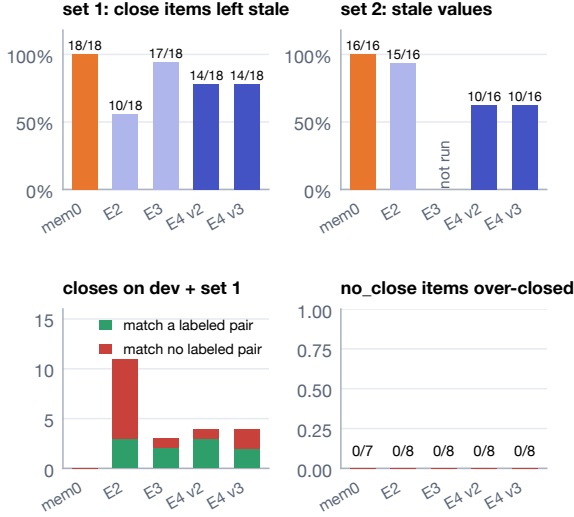


Figure 4: Store outcomes on the update sets: set-1 close items left stale, set-2 stale values, closes on dev + set 1 split by whether they match a labeled pair, and no_close items over-closed. E3 was not run on set 2.

no labeled pair. On set 2 the comparison is 10 of 14 against 1 of 10.

Stale items. A stale item is a close item whose old fact is still active. Belief v2 left 14/18 of the set-1 close items stale, against 17/18 under belief v1 and 18/18 for mem0, which never closes. On set 2 it left 10/16 stale, against 15/16 and 16/16.

Fulfilled plans. A relaxed heuristic closed a plan whenever a related past or current fact arrived. Across its two arms it fired 25 times. 1 firing matched a labeled (plan, fulfilment) pair, and 24 did not. The frozen arm uses the plan_fulfilled question instead; it was asked 234 times on dev + set 1 and closed 2 plans correctly and 1 wrongly.

Belief v3. On the same data, v3 ignored 36 weak answers on dev + set 1 and 39 with set 2 added. It closed the same four facts as v2 on dev + set 1. One of them, D2:5, crossed the close line at a different message (U:S01 rather than U:E11). Since that pair is not labeled, the audit scores v3 at 2 matching and 2 not matching, against v2’s 3 and 1. Stale and over-close counts are unchanged. v3 targets the weak-support failure that closed true facts under Laya (§5.4).

Hygiene. One pass cost \$0.0015 on dev + set 1 (235 decisions). On the held-out conversations it cost \$0.0112, \$0.0254, \$0.0332 and \$0.0323, making 9, 23, 32 and 24 links.

Contradiction regression. The regression set has 50 contradiction pairs, each an old fact, a new fact and a message, labeled with accepted relations

and a temporal status (bench/contradiction_pairs.jsonl). Under the current question versions, Jev’s relation choice is in the accepted set for 90.0% of pairs (easy 100.0%, medium 93.3%, subtle 73.3%), and its temporal status is right for 94.0%. The write path’s close rule is right for 76.0%, with 0 false closes. Jev’s mean top probability is 0.88 when it is right and 0.81 when it is wrong.

Calibration error. We measure expected calibration error (ECE) on two label sets: the 50 gold pairs, and 29 escalation labels, where Sonnet decided a relation Jev was unsure of. On the gold pairs Jev’s relation ECE is 0.14 (fitted $T = 1.48$) and its temporal ECE is 0.04. On the escalation labels, which are by construction the cases Jev was unsure of, relation ECE is 0.15. Table 5 has the full comparison, with Laya, and Figure 6 in Appendix C the reliability diagrams.

Cost/error tradeoff. The 29 escalation labels are too few for a curve, so Figure 5 uses the 50 gold pairs. The Jev cost $c_J = \$0.000016$ is the mean cost of one relation decision over 36,429 held-out decisions, and the escalation cost $c_L = \$0.0073$ is the mean over 148 logged escalations. No LLM run on the gold pairs is saved, so ε_L is an assumption, plotted at 0 and 0.1. With Jev alone the error is 10.0%. At $\theta = 0.85$, 32.0% of decisions escalate, the cost is \$0.002368 per decision, and E is 6.0% ($\varepsilon_L = 0$) or 9.2% ($\varepsilon_L = 0.1$). At the production threshold of 0.60, 8.0% escalate and E is 10.0% or 10.8%.

5.4 Negative results

Closes do not change answers. The accuracy columns of Table 4 show it. mem0 never closes anything and leaves every set-1 close item stale (18/18), yet answers 29/30 set-1 questions at default k . The arm that closes most aggressively leaves 10/18 stale and answers 30/30.

The answer model resolves recency itself: extraction writes dates into the memory text, and the compact rendering adds the date each fact was said. Removing all dates, validity and source from the answer context (the no-dates column) still leaves every arm we ran that way at 29/30 or higher on set 1. Only the $k=3$ budget separates the systems (25/30 for mem0, 30/30 for engram), and §5.2 attributes that to ranking, not to closes.

Set 2, which asks about chains of changes and past moments, is at its ceiling for every arm. Current LoCoMo-style questions do not reward a cor-

Table 5: Calibration on the escalation labels (dev + update sets, labels by claude-sonnet-4-6) and the gold contradiction pairs; Jev jev-1.13.0, Laya base checkpoint zero-shot. ECE uses 10 equal-width bins on the top choice. T is fitted by NLL per question. The last column is out of sample (2-fold). Relation probabilities fold negatives into contradiction, since the pairs predate negatives.

labels	question	backend	n	top-1 acc.	mean conf.	ECE	fitted T	ECE at T
escalation	relation	jev	29	79.3%	0.92	0.15	2.51	0.10
escalation	relation	laya	29	10.3%	0.47	0.37	5.41	0.12
gold pairs	relation	jev	50	82.0%	0.87	0.14	1.48	0.14
gold pairs	relation	laya	50	28.0%	0.47	0.20	4.22	0.02
gold pairs	temporal	jev	50	94.0%	0.95	0.04	1.13	0.04
gold pairs	temporal	laya	50	76.0%	0.72	0.06	0.74	0.07

Table 6: Contradiction regression (50 gold pairs), relation_to_candidate and temporal_status in one request. Laya: convaiinnovations/laya base checkpoint, zero-shot, fp16, on the local MLX server.

backend	relation exact	temporal	close rule	closes	false closes
Jev (jev-1.13.0)	90.0%	94.0%	76.0%	22	0
Laya, Jev wording (truncated)	48.0%	76.0%	32.0%	0	0
Laya, native wording (fits)	38.0%	76.0%	32.0%	0	0



Figure 5: $C(\theta)$ and $E(\theta)$ on the 50 gold pairs. ε_L is assumed, not measured; the 29 escalation labels were too few for a curve.

rect store.

A small model used zero-shot. Everything in this part concerns the 421M base checkpoint, used zero-shot as its documentation advises against (Convai Innovations, 2026). The checkpoint fine-tuned for typed decisions (reported at 0.766 on its own benchmark) and fine-tuning on our escalation labels were not tested; they are the obvious follow-up.

On the 50 regression pairs (Table 6), Laya

chooses an accepted relation for 48.0% with Jev’s question wording and 38.0% with wording rewritten to fit its 192-token question budget. Jev scores 90.0%. Laya never reaches the action threshold, so it closes nothing. Its gold-pair relation accuracy is 28.0% (Table 5). A temperature lowers its ECE but not its accuracy.

With the frozen Jev arm replayed from cache and Laya answering every request on the side, the two gave the same answer on 44.8% of 6,764 decisions and the same action at the 0.85 threshold on 31.2%. On relation_to_candidate they agreed on 5.9% (Table 12 and Figure 7, Appendix C).

When Laya decided every question, on dev + set 1 at $k=3$ it scored 24/35 LoCoMo and 26/30 update questions. Jev scored 29/35 and 30/30. It made 23 closes, of which 23 match no labeled pair, and left 43 of 66 facts active. Most of these closes came from weak duplicate and refinement answers below 0.5 lowering belief, the failure belief v3 removes (§3.3).

Hybrid. In a hybrid arm Laya answered the two high-volume yes/no questions (relevant_to_query, same_fact) and Jev the rest. On the routed questions the two models gave the same answer on 86.9% and the same action on 34.1%. Writes were identical to the all-Jev arm, but at $k=3$ only 51.8% of Jev’s top-3 lines survived the change of reranker. The hybrid saved 42.6% of Jev’s cost on dev + set 1 (\$0.025 per run); Table 13 in Appendix C has its accuracy.

5.5 Systems notes

Jev latency is flat in request size. Across 9,446 live Jev requests from 25 runs, median latency is between 222 ms and 269 ms for requests of 1 to 50 questions, and 371 ms for 51–80. A least-squares fit gives 380 ms plus 7.19 ms per question (Figure 8 and Table 14, Appendix C). Variation over time is larger than variation over size. For requests of 16–20 questions, the per-run median was between 212 ms and 267 ms in 12 runs, and 793 ms and 607 ms in the two held-out runs made during one slower period.

Extraction dominates write cost. On the held-out set, engram’s write cost is \$10.22 to \$10.55 per 1,000 messages and mem0’s is \$9.92 to \$10.16. The decision layer is 3.3% to 4.8% of engram’s (Table 15, Appendix C).

6 Discussion and Limitations

What the decision layer buys, and what it does not. Typed decisions buy lower cost and latency (§5.1), an audit trail in which every decision carries probabilities and a question version, store operations that are reversible and gated (§5.3), and a price at which re-examining the store is routine (§3.2). They do not buy accuracy at default k , where the systems tie (§5.1), or accuracy on today’s update questions, where every arm is near its ceiling (§5.4).

Benchmarks do not see storage correctness. A question about a fact that changed is answerable from a store that kept both versions, as long as the memories carry dates. An evaluation that rewards a correct store would score answers against validity windows, use small retrieval budgets in which a stale fact displaces a current one, store memories without dates, and ask about long chains of changes. Update set 2 targets the last of these, and every arm is at its ceiling on it.

Rerank or larger k . Under a three-memory budget, a cheap listwise rerank is worth +8.7 points over mem0 at matched tokens. At $k=20$ the ranking stops mattering. For deployments that pay per context token, reranking is the cheaper route to the same accuracy.

Limitations. The comparison has one baseline, mem0 OSS 2.1.0, on one benchmark, four held-out LoCoMo conversations. The update sets and the regression pairs were written and labeled by the system’s author. Jev is the only decision model evaluated as the deciding backend, at one version,

and Laya was tested only as a zero-shot base checkpoint. mem0’s open-source path gives no observation date, so relative dates resolve against the run date; we use it as shipped, and a variant with the session date patched in scored 31/35 on dev against 30/35 unpatched, within noise. The attribution of the matched-context gain to reranking (§5.2) rests on a dev ablation, not a held-out one. Finally, the judge is an LLM (claude-sonnet-4-6), and we did not measure its agreement with human labels.

7 Conclusion

With extraction held identical, typed decisions replaced an LLM decision layer at $70.0\times$ lower cost and $27.6\times$ lower median decision latency, with no measured accuracy loss. On 610 held-out questions, a cheap listwise rerank gave +8.7 points over mem0 at matched context under a three-memory budget, and the systems tied at $k=20$. Closing stale facts, the part of the design aimed at correctness, did not change answers on current benchmarks. The base checkpoint of a small open-weights decision model, used zero-shot, did not make the relational decisions.

References

- Anastasios N. Angelopoulos and Stephen Bates. 2021. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*.
- ByteRover. 2026. Benchmarking AI agent memory: ByteRover 2.0 scores 92.2% and rewrites the LoCoMo leaderboard. <https://byterover.dev/blog/benchmark-ai-agent-memory>. Accessed September 2026.
- Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*. ArXiv:2305.05176.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Convai Innovations. 2026. Laya: a non-autoregressive system 1 decision model. Hugging Face model card, <https://huggingface.co/convaiinnovations/laya>. ModernBERT-large, 421M parameters. Accessed September 2026.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th Interna-*

- tional Conference on Machine Learning (ICML)*. ArXiv:1706.04599.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*.
- Dongming Jiang, Yi Li, and Bingzhe Li. 2026a. Jev-Mem: System-one-controlled agentic memory for efficient AI agents. *arXiv preprint arXiv:2609.23986*. Concurrent work, posted 21 September 2026.
- Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. 2026b. MAGMA: A multi-graph based agentic memory architecture for AI agents. *arXiv preprint arXiv:2601.03236*.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173. ArXiv:2307.03172.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. [Evaluating very long-term conversational memory of LLM agents](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand. Association for Computational Linguistics. ArXiv:2402.17753.
- Mem0. 2026. LoCoMo vs. LongMemEval vs. BEAM: The 2026 AI memory benchmark guide. <https://mem0.ai/blog/ai-memory-benchmarks-in-2026>. Accessed September 2026.
- Andy Nguyen, Danh Doan, Hoang Pham, Bao Ha, Dat Pham, Linh Nguyen, Hieu Nguyen, Thien Nguyen, Cuong Do, Phat Nguyen, and Toan Nguyen. 2026. ByteRover: Agent-native memory through LLM-curated hierarchical context. *arXiv preprint arXiv:2604.01599*.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to route LLMs with preference data. *International Conference on Learning Representations (ICLR)*. ArXiv:2406.18665.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*. The system now maintained as Letta.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.
- Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of EMNLP*. ArXiv:2304.09542.
- Javad Taghia. 2026. Governed agent memory with structured judgment: An AtMem-Jev retrieval study. Hugging Face community article, <https://huggingface.co/blog/javadtghia/an-atmem-and-jev-experiment-can-a-judgment-model-h>. Published 19 September 2026.
- TypeSafe AI. 2026. Jev: introduction and the choice primitive. <https://docs.typesafe.ai/introduction>. Accessed September 2026.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.10813.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*.

A Decision questions

Table 7 lists the 12 questions in the current chain with their types and options. `edge_type` and `query_relation` choose among 25 relation types. The instructions and the rubric for every option of every version are in `src/engram/decide/questions.py`, and the reasons for each version change are in `docs/DECISIONS.md`.

Table 7: Jev questions in the current chain (src/engram/decide/questions.py).

question	type	options
worth_remembering	noul v1	yes / no
fact_kind	choice v1	preference, bio, event, relationship, task, opinion
temporal_status	choice v2	current, planned, past, hypothetical
relation_to_ candidate	choice v2	new, duplicate, update, contradiction, refinement, negates
relation_to_ candidate_recheck	choice v2	new, duplicate, update, contradiction, refinement, negates
edge_type	choice v1	25 relation types
durability	choice v1	permanent, long_term, short_ lived
sensitivity	choice v1	none, health, financial, relationship, credentials
plan_fulfilled	noul v1	yes / no
relevant_to_query	noul v1	yes / no
query_relation	choice v1	25 relation types
same_fact	noul v1	yes / no

B Update sets

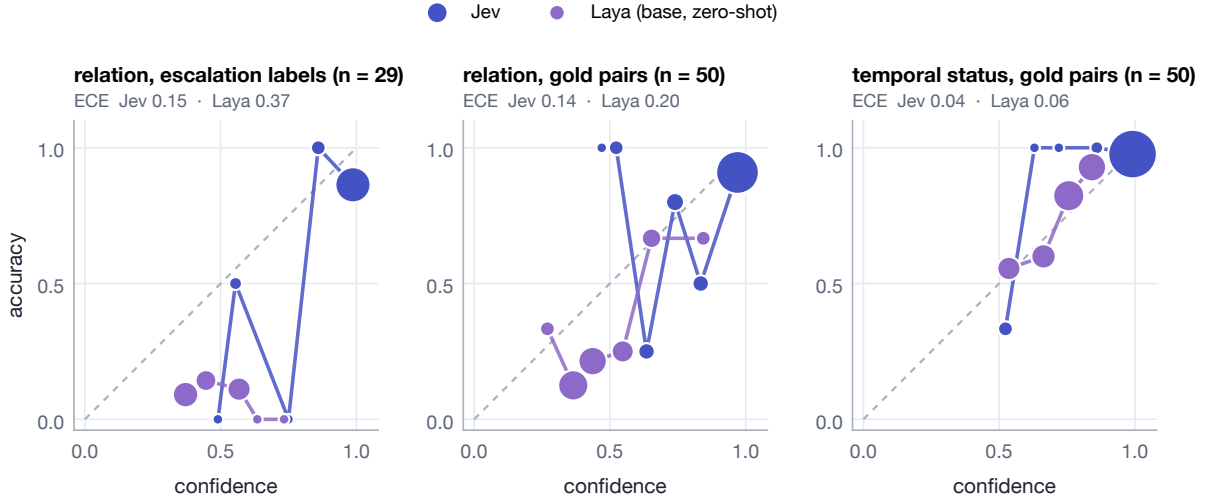
Table 8: One item of each kind from the two update sets (bench/updates_conv26.json, bench/updates2_conv26.json).

id	kind	earlier fact or message	update message(s)	question	gold
E01	set 1, easy close	Melanie’s main creative outlet is painting, which relaxes her after a long day.	I haven’t painted in weeks, honestly. Pottery is my main creative outlet now - I go to the studio three evenings a week.	What is Melanie’s main creative outlet now?	Pottery (she has stopped painting)
S01	set 1, subtle (no_close)	Melanie has been married to her husband for 5 years.	Our first apartment after the wedding was so tiny - we still laugh about it.	Is Melanie still married?	Yes, married for 5 years
F01	set 1, fulfilled plan	Caroline plans to continue her education.	I did it - I enrolled in a psychology certificate program at the community college. Classes start next week!	Has Caroline enrolled in a program to continue her education?	Yes, a psychology certificate program at the community college
C01q1	set 2, chain	I just moved into a studio apartment downtown.	Moved again! I’m in a two-bedroom in Oak Park now, so there’s room for a foster kid. → We finally found a house - I live in Evanston now.	Where does Caroline live now?	A house in Evanston
N01	set 2, no temporal cue	My favorite coffee spot is Blue Door Café.	My favorite coffee spot is Grind House.	What is Caroline’s favorite coffee spot?	Grind House

C Per-conversation and systems tables

Table 9: Held-out accuracy per conversation, $k=3$. Q per row; models as in Table 2.

conv.	Q	engram	mem0	Δ (questions)	engram tokens / q	mem0 tokens / q
conv-30	81	59/81	55/81	4	285	154
conv-41	152	113/152	89/152	24	297	161
conv-42	199	150/199	111/199	39	292	163
conv-43	178	125/178	101/178	24	283	153

**Figure 6:** reliability diagrams (accuracy against confidence) for Jev and Laya on both label sets.**Table 10:** Held-out accuracy per conversation, engram $k=3$ against token-matched mem0 ($k=6$). Q per row; models as in Table 2.

conv.	Q	engram $k=3$	mem0 $k=6$	Δ (questions)
conv-30	81	59/81	54/81	5
conv-41	152	113/152	109/152	4
conv-42	199	150/199	119/199	31
conv-43	178	125/178	112/178	13

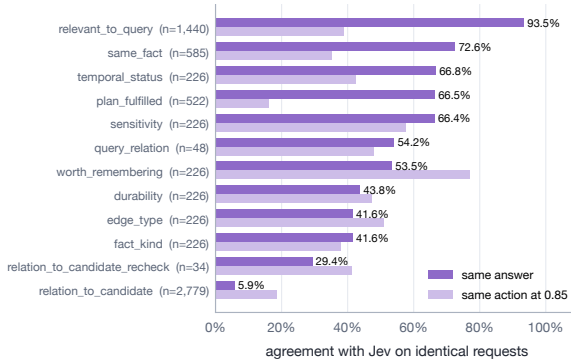
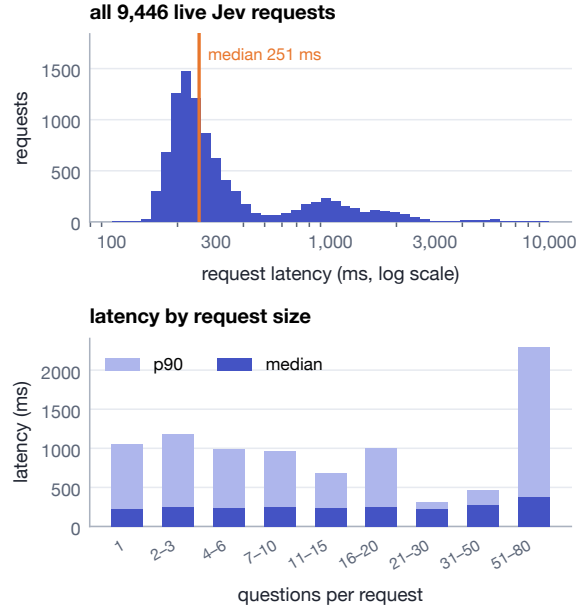
**Figure 7:** Agreement between the Laya base checkpoint (zero-shot) and Jev on identical requests, per question, sorted by same answer. Dev + update sets 1 and 2, frozen arm's trajectory.**Figure 8:** Jev request latency: distribution over all logged requests, and median and p90 by request size.

Table 11: Held-out accuracy per conversation, $k=20$. Q per row; models as in Table 2.

conv.	Q	engram	mem0	Δ (questions)	engram tokens / q	mem0 tokens / q
conv-30	81	62/81	63/81	-1	1,434	996
conv-41	152	128/152	135/152	-7	1,521	1,049
conv-42	199	156/199	146/199	10	1,491	1,043
conv-43	178	136/178	133/178	3	1,389	995

Table 12: Laya against Jev on identical requests: dev + update sets 1 and 2, frozen arm’s trajectory at $k=3$ (write and retrieval decisions; n per row). Jev jev-1.13.0; Laya base checkpoint, zero-shot. Same action: both choose the same label at $p \geq 0.85$, or neither reaches it. Acts: share of decisions at $p \geq 0.85$.

question	n	same answer	same action	Jev acts	Laya acts
relation_to_candidate	2,779	5.9%	18.5%	79.9%	6.4%
relevant_to_query	1,440	93.5%	38.9%	89.7%	31.3%
same_fact	585	72.6%	35.2%	96.2%	34.4%
plan_fulfilled	522	66.5%	16.1%	90.0%	32.6%
worth_remembering	226	53.5%	77.0%	18.1%	4.9%
fact_kind	226	41.6%	38.1%	63.7%	15.5%
temporal_status	226	66.8%	42.5%	62.8%	5.3%
edge_type	226	41.6%	50.9%	29.2%	55.3%
durability	226	43.8%	47.3%	52.7%	0.0%
sensitivity	226	66.4%	57.5%	45.1%	7.1%
query_relation	48	54.2%	47.9%	39.6%	75.0%
relation_to_candidate_ recheck	34	29.4%	41.2%	35.3%	26.5%

Table 13: Hybrid against all-Jev: dev slice with update sets (Q per row: 35 LoCoMo, 30 set 1, 20 set 2), k as labeled, extraction claude-haiku-4-5, answers and judge claude-sonnet-4-6.

questions	all-Jev $k=3$	hybrid $k=3$	all-Jev $k=20$	hybrid $k=20$
update set 1	30/30	27/30	30/30	30/30
LoCoMo dev	29/35	29/35	31/35	31/35
update set 2	19/20	17/20	20/20	20/20

Table 14: Jev (jev-1.13.0) latency by request size over all logged runs (dev, stress and held-out slices), from the decision logs (bench/results/jev_latency.json). Client-measured, after the rate limiter, retries included.

questions / request	requests	median input tokens	median latency	p90 latency
1	1,305	564	229 ms	1,052 ms
2–3	1,054	870	248 ms	1,176 ms
4–6	775	1,562	231 ms	992 ms
7–10	1,417	4,314	246 ms	967 ms
11–15	227	2,432	231 ms	680 ms
16–20	3,240	5,593	255 ms	1,009 ms
21–30	184	3,551	222 ms	312 ms
31–50	505	6,903	269 ms	472 ms
51–80	739	9,187	371 ms	2,290 ms

Table 15: Held-out write side (conv-30, 41, 42, 43; writes only). Extraction claude-haiku-4-5 for both systems; decisions Jev jev-1.13.0 with claude-sonnet-4-6 escalations. One ingestion per system per conversation. Decision layer = Jev plus escalations.

conv.	engram \$/1k	decision \$/1k	mem0 \$/1k	decision p50	engram write p50	mem0 write p50	engram facts	mem0 memories
conv-30	\$10.22	\$0.33	\$9.92	1,982 ms	928 ms	1,034 ms	267	276
conv-41	\$10.54	\$0.47	\$10.16	1,614 ms	1,775 ms	1,437 ms	668	822
conv-42	\$10.55	\$0.50	\$10.04	981 ms	1,824 ms	1,300 ms	708	695
conv-43	\$10.45	\$0.50	\$9.96	868 ms	1,819 ms	1,210 ms	628	636

D Reproduction

Each table and figure was produced by the command below, run from the root of the engram codebase. With the call cache (bench/.cache/calls.sqlite) in place, every command in Table 16 replays at no API cost.

The spend ledger records cost per run, not per table; Table 17 gives per-arm totals.

Table 16: The command behind each table and figure.

table / figure	command
Table 1	python -m bench.run --arm {e2_jev,e2_llm,mem0} --slice dev
Tables 2, 3, 9–11; Fig. 2	python -m bench.run --arm {e4_belief_v2,mem0} --slice heldout:<conv> --top-k 3 --also-top-k 20, then python -m bench.heldout_report and python -m bench.token_match
Table 4; Figs. 3, 4	python -m bench.run --arm <arm> --slice dev_updates[2] [--top-k 3] [--no-dates]
Tables 5, 6; Figs. 6, 7	python bench/test_contradictions.py --backend laya [--native] --save . . . , python -m bench.calibration, python -m bench.jev_regression (the Laya rows need bench/laya_server.py running)
Fig. 5	python -m bench.tradeoff
Tables 12, 13	python -m bench.laya_report, python -m bench.hybrid_report
Table 14, Fig. 8	python -m bench.jev_latency
this paper	python paper/build.py, python paper/figures.py

Table 17: Phase 2 spend per arm, in USD (bench/
results/phase2_spend.jsonl).

arm	runs	Jev	Claude
mem0	15	\$0.0000	\$35.94
e4_belief_v2	18	\$1.0917	\$31.80
e2_jev	2	\$0.0636	\$3.51
e2_llm	2	\$0.0134	\$3.21
mem0_token_ matched	1	\$0.0000	\$2.52
e4_belief_v2_ laya	2	\$0.0405	\$2.40
e2_jev_v2	4	\$0.0362	\$2.35
e1_recall	4	\$0.0653	\$2.05
e4_belief	6	\$0.0316	\$1.69
e3_structural_v2	1	\$0.0278	\$1.44
e0_baseline	8	\$0.0736	\$1.36
mem0_dated	1	\$0.0000	\$0.96
e3_structural_v3	2	\$0.0272	\$0.76
e4_belief_v2_ hybrid	4	\$0.0000	\$0.73
e4_belief_v2_ compact	2	\$0.0000	\$0.59
e4_belief_v2_ norerank	1	\$0.0000	\$0.25
e4_belief_v2_ shadow	2	\$0.0000	\$0.18
e3_structural	1	\$0.0008	\$0.04
e4_belief_v2_ nohist	1	\$0.0000	\$0.01
e4_belief_v3	3	\$0.0004	\$0.00